

UNIVERSIDAD DE LOS ANDES
Facultad de Ingeniería y Ciencias Aplicadas
Arquitectura de Computadores 2026-20

Entrega Nº2

Juego de reflejos en assembly sobre Pochoco SoC

Grupo 2:
Josefa Alcaíno Lira
Felipe Izcúe Ferrer
Martín Norton Haigis

Profesor de Cátedra: Jorge Gómez M.

Ayudantes:
Chen Shi Ye
Nicolás Villegas

13 de octubre de 2026

Resumen

En este informe se documenta un juego de reflejos escrito en assembly RV32E que corre sobre el procesador Espino del Pochoco SoC, instanciado en una FPGA Lattice iCE40 HX1K (Nandland Go Board). En cada ronda el juego enciende los cuatro LEDs durante al menos tres segundos, elige un LED objetivo de forma pseudoaleatoria, mide el tiempo que tarda el jugador en apretar el botón correspondiente y lo muestra en décimas de segundo en los dos displays de siete segmentos. Tras diez respuestas correctas muestra el promedio. Para medir tiempo se agregó al SoC un contador de ciclos de 32 bits mapeado en memoria (`0x8000000C`). El programa se traduce a código de máquina con un assembler propio escrito en Python, que se verificó contra el assembler de GNU, contra el decodificador del procesador y mediante simulación del juego completo. Se corrigió además un defecto del reset de encendido del SoC original que solo aparecía en la placa.

Índice

1. Introducción **4**
1.1. Reglas del juego 4

2. Arquitectura del sistema **5**
2.1. Visión general 5
2.2. Pochoco SoC y procesador Espino 5
2.3. Modificaciones al RTL 6
2.4. Recursos y frecuencia 8

3. Temporización y medición de tiempo **8**
3.1. Frecuencia de reloj, unidad de tiempo y conversión 8
3.2. Espera de tres segundos 9
3.3. Rebotes de los botones 10
3.4. Medición del tiempo de respuesta 10
3.5. Promedio 11

4. Generación pseudoaleatoria **11**

5. El programa en assembly **13**
5.1. Estructura 13
5.2. Uso de registros 14
5.3. Aritmética sin multiplicación, división ni desplazamientos 14
5.4. Instrucciones utilizadas 15

6. El assembler **16**
6.1. Sintaxis aceptada 16
6.2. Tratamiento de etiquetas: tres pasadas 18
6.3. Codificación e inmediatos 18
6.4. Validación 19
6.5. Ejemplos de traducción 20

7. Verificación **21**
7.1. Pruebas del assembler 21
7.2. Prueba de instrucciones sobre el procesador 21
7.3. Simulación del juego 21
7.4. Simulación post-síntesis 22
7.5. Prueba en la placa: el reset de encendido 23

8. Compilación, programación y cambio de constantes **23**

9. Conclusiones **25**

Declaración de Uso de Herramientas de Inteligencia Artificial **27**

1. Introducción

El objetivo del proyecto es programar en assembly un juego de reflejos que corra sobre un procesador RISC-V real instanciado en una FPGA, y traducir ese programa a código de máquina con un assembler desarrollado por el grupo, sin invocar ningún assembler RISC-V externo. El punto de partida es el Pochoco SoC [2]: un sistema en chip con el procesador Espino (RV32E), una RAM unificada para instrucciones y datos, y periféricos mapeados en memoria para los LEDs, los botones y los displays de la Go Board [5].

El trabajo tiene tres partes: el **hardware** (sección 2), el **programa del juego** (secciones 3, 4 y 5) y el **assembler** (sección 6). La sección 7 describe cómo se verificó cada parte y la sección 8 cómo compilar, programar la placa y cambiar una constante.

1.1. Reglas del juego

1. Al encender, los LEDs parpadean alternando 0101 y 1010 y los displays muestran `00`. El juego espera que se apriete cualquier botón.

2. Cada ronda comienza con los cuatro LEDs encendidos durante al menos tres segundos.

3. Luego queda encendido solo el LED objetivo, elegido de forma pseudoaleatoria, y en ese mismo instante comienza la medición de tiempo.

4. El jugador aprieta un botón. Si es el del LED objetivo, los displays muestran el tiempo de respuesta en décimas de segundo y la ronda cuenta como correcta.

5. Si es otro botón, los displays muestran `EE` y la ronda actual se repite. Las rondas correctas acumuladas se conservan.

6. Tras diez respuestas correctas, el último tiempo queda visible tres segundos y después se muestra el promedio, con los LEDs parpadeando. Un botón inicia una partida nueva.

2. Arquitectura del sistema

2.1. Visión general

La figura 1 muestra el sistema. El procesador ejecuta el programa que está en la RAM y se comunica con el mundo exterior solo mediante lecturas y escrituras a direcciones de memoria: el bit 31 de la dirección distingue la RAM de los periféricos.

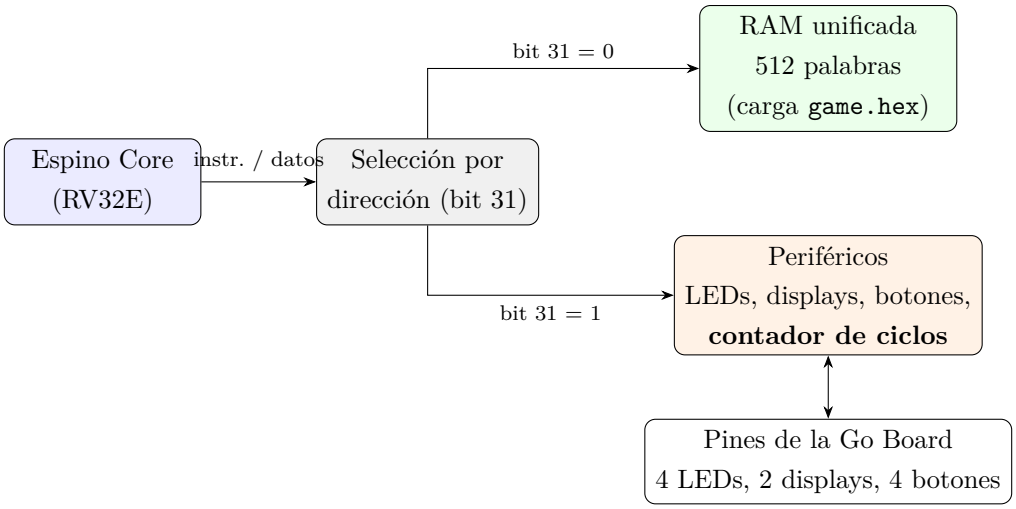


Figura 1: Diagrama de bloques del sistema.

2.2. Pochoco SoC y procesador Espino

El procesador Espino implementa RV32E [1]: el conjunto base de 32 bits de RISC-V con solo 16 registros (`x0` a `x15`). No incluye la extensión M, es decir, no tiene multiplicación ni división. Además, el ALU del repositorio tiene los desplazamientos comentados para ahorrar celdas lógicas: `sll`, `srl`, `sra` y sus variantes con inmediato se decodifican, pero se ejecutan como una suma. Estas dos restricciones determinan cómo se escribió el programa (sección 5.3).

El core tiene dos etapas: búsqueda de instrucción (IF) y decodificación-ejecución (ID). El banco de registros está hecho con bloques de RAM de la FPGA, cuya lectura es síncrona, así que cada instrucción espera un ciclo por sus operandos. De ahí se deducen los costos de la tabla 1: las cargas esperan un ciclo más por el acceso a memoria, y un salto tomado deja una burbuja en la etapa IF. Estos valores se usan en la sección 3 para acotar el error de medición.

Cuadro 1: Ciclos de reloj por instrucción del Espino.

Instrucción	Ciclos
ALU (`add`, `addi`, `xor`, ...), `lui`, `sw`	2
Branch no tomado	2
`lw` (RAM o periférico)	3
Branch tomado, `jal`, `jalr`	3

Fuente: Elaboración propia.

La RAM tiene 512 palabras (2 KB) y guarda a la vez el programa y los datos. Se inicializa al sintetizar con el contenido de un archivo `.hex` mediante `$readmemh`, y el procesador comienza a ejecutar en la dirección 0. La tabla 2 muestra el mapa de memoria que usa el juego.

Cuadro 2: Mapa de memoria. La última fila es el registro agregado por el grupo.

Dirección	Acceso	Contenido
`0x0000_0000`	lectura/escritura	RAM: programa y datos (512 palabras)
`0x8000_0000`	escritura	displays: bits 7:4 izquierdo, bits 3:0 derecho
`0x8000_0004`	escritura	LEDs, bits 3:0
`0x8000_0008`	lectura	botones, bits 3:0 (1 = apretado)
`0x8000_000C`	lectura	**contador de ciclos, 32 bits**

Fuente: Elaboración propia.

Cada display muestra un dígito hexadecimal: el periférico traduce los 4 bits a los siete segmentos. Para mostrar un número decimal de dos cifras el programa escribe decenas $\cdot$ 16 + unidades; por ejemplo, para 37 escribe `0x37`.

2.3. Modificaciones al RTL

Se hicieron tres cambios al hardware, todos visibles con `git diff` desde el primer commit del repositorio, que es el código original sin modificar.

Contador de ciclos. El juego necesita medir tres segundos y tiempos de reacción. Un bucle de espera calibrado no sirve, porque su duración depende de qué instrucciones ejecuta (tabla 1) y cambiaría con cualquier modificación del programa. Se agregó a `rtl/pochoco_periph.v` un registro de 32 bits que suma uno en cada flanco de reloj y que el programa lee en el desplazamiento `0x0C` del bloque de periféricos (código 1). Es de solo lectura, comienza en cero tras el reset y da la vuelta cada 2^{32} ciclos, unos 171,8 segundos.

```
reg [31:0] cycle_q;
always @(posedge clk_i or negedge rst_ni) begin
  if (!rst_ni) cycle_q <= 32'b0;
  else         cycle_q <= cycle_q + 32'd1;
end
...
    case (off)
      6'd2: rdata_o <= {28'b0, btn_i}; // Buttons
      6'd3: rdata_o <= cycle_q;        // Cycle counter   <- nuevo
```

Código 1: Contador de ciclos agregado a `pochoco_periph.v`.

Módulo superior `game_top`. El archivo `rtl/game_top.v` es el que se sintetiza y programa en la FPGA. Tiene los mismos puertos que la Go Board según `goboard.pcf` e instancia el SoC con `MemFile` apuntando a `sw/game.hex`. El botón i corresponde al LED i: `i_Switch[i]` y `o_LED[i]`. `MemFile` es además un parámetro del módulo para que la simulación post-síntesis pueda cargar la versión del juego con tiempos cortos (sección 7).

```
module game_top #(
  parameter MemFile = "sw/game.hex"
) (
  input  wire       i_Clk,        // 25 MHz
  output wire [3:0] o_LED,
  input  wire [3:0] i_Switch,     // boton i corresponde al LED i
  ... // displays y SPI, con los nombres de goboard.pcf
);
  pochoco_soc #(.MemFile(MemFile)) u_pochoco ( .i_Clk(i_Clk), ... );
endmodule
```

Código 2: Instancia del SoC en `game_top.v` (puertos abreviados).

Reset de encendido. En `rtl/pochoco_soc.v`, el contador que mantiene al procesador en reset después de configurarse la FPGA pasó de 4 a 12 bits: el procesador arranca a los 4.095 ciclos (164 μs) en vez de a los 15 (0,6 μs). Con el valor original el juego no funcionaba en la placa aunque todas las simulaciones pasaban. La sección 7.5 describe cómo se encontró la causa.

```
reg  [11:0] por_cnt = 12'b0;          // antes: 4 bits, hasta 4'hF
wire        rst_ni;
always @(posedge clk) begin
  if (por_cnt != 12'hFFF) por_cnt <= por_cnt + 12'd1;
end
assign rst_ni = (por_cnt == 12'hFFF);
```

Código 3: Reset de encendido en `pochoco_soc.v`.

Lo que no se cambió. No se activaron los desplazamientos del ALU: el diseño ya usa el 91 % de las celdas lógicas y un desplazador de 32 bits cuesta del orden de cien celdas. El juego se escribió sin desplazamientos (sección 5.3). El módulo SPI del SoC original queda sin uso.

2.4. Recursos y frecuencia

El diseño final, sintetizado con Yosys y nextpnr [6], usa 1173 de las 1280 celdas lógicas (91 %) y 12 de los 16 bloques de RAM (75 %). nextpnr reporta una frecuencia máxima de 42,8 MHz, frente a los 25 MHz del reloj de la placa. El Makefile ejecuta nextpnr con `-freq 25`, así que la compilación falla si alguna vez no se alcanzara esa frecuencia.

3. Temporización y medición de tiempo

3.1. Frecuencia de reloj, unidad de tiempo y conversión

La Go Board tiene un oscilador de 25 MHz conectado al pin 15 de la FPGA, y el SoC usa ese reloj sin dividirlo. **La unidad de tiempo del juego es la décima de segundo.** Todas las cantidades de tiempo se derivan de la frecuencia:

$$f_{\text{clk}} = 25\,000\,000 \text{ Hz}, \qquad T_{\text{clk}} = \frac{1}{f_{\text{clk}}} = 40 \text{ ns}$$

$$N_{\text{décima}} = f_{\text{clk}} \cdot 0{,}1 \text{ s} = 2\,500\,000 \text{ ciclos}$$

$$N_{3\,\text{s}} = f_{\text{clk}} \cdot 3 \text{ s} = 75\,000\,000 \text{ ciclos}$$

Si el contador marca c_0 cuando se enciende el LED objetivo y c_1 cuando se detecta el botón, el tiempo de respuesta mostrado es

$$t = \left\lfloor \frac{(c_1 - c_0) \bmod 2^{32}}{2\,500\,000} \right\rfloor \text{ décimas.}$$

- **La resta se hace sin signo, en 32 bits** (`sub` da el resultado módulo 2^{32}). Si el contador dio la vuelta entre c_0 y c_1, la resta sigue dando la cantidad correcta de ciclos. Solo falla una espera de más de 2^{32} ciclos, unos 171,8 segundos.
- **La división trunca**, como un cronómetro: 0,29 s se muestra como `02`. Un valor mostrado de n décimas significa que el tiempo real está entre n y $n+1$ décimas.
- **La división se hace por restas sucesivas** (subrutina `divu`), porque el procesador no tiene instrucción de división. Cada vuelta del bucle toma 9 ciclos (`bltu` no tomado 2, `sub` 2, `addi` 2, `jal` 3), así que convertir 9,9 s (99 vueltas) toma unos 900 ciclos, 36 μs. Esa cuenta ocurre después de leer c_1, así que no afecta la medición.

Las constantes están con nombre al comienzo de `game.s` (tabla 3). Cambiar la frecuencia del reloj solo exige cambiar `WAIT_CYCLES`, `TENTH_CYCLES`, `DEBOUNCE_CYCLES` y `BLINK_CYCLES`.

Cuadro 3: Constantes del juego en `sw/game.s`.

Constante	Valor	Significado
`ROUNDS`	10	respuestas correctas por partida
`WAIT_CYCLES`	75 000 000	fase con los 4 LEDs: $3 \cdot 25\,000\,000$ ciclos
`TENTH_CYCLES`	2 500 000	ciclos en una décima de segundo
`DEBOUNCE_CYCLES`	500 000	20 ms sin botones apretados antes de mostrar el objetivo
`BLINK_CYCLES`	6 250 000	0,25 s de cada estado del parpadeo
`ERROR_CODE`	`0xEE`	lo que muestran los displays ante un botón incorrecto

Fuente: Elaboración propia.

3.2. Espera de tres segundos

La subrutina `wait_cycles` (código 4) lee el contador al entrar y vuelve cuando la diferencia con la lectura actual alcanza el valor pedido en `a0`. La comparación `bltu` es sin signo, por la misma razón que la resta.

```
wait_cycles:
    lw   t0, CYCLES(s0)            # t0 = inicio
wc_loop:
    lw   t1, CYCLES(s0)
    sub  t1, t1, t0                # t1 = ciclos transcurridos
    bltu t1, a0, wc_loop
    ret
```

Código 4: Espera de a0 ciclos.

Cada vuelta toma 8 ciclos (`lw` 3, `sub` 2, `bltu` tomado 3), así que la espera termina a lo más 8 ciclos (320 ns) después de cumplirse los 75.000.000. Tras ella el programa exige además 20 ms seguidos sin ningún botón apretado (siguiente subsección). La fase con los cuatro LEDs dura entonces al menos 3,02 s, lo que cumple el "al menos tres segundos" del enunciado.

3.3. Rebotes de los botones

Un botón mecánico rebota: al apretarlo o soltarlo, el contacto se abre y cierra varias veces durante algunos milisegundos. El juego solo necesita el primer contacto para detener el cronómetro, así que los rebotes al apretar no afectan la medición. Los rebotes al soltar sí son un problema: si el jugador suelta el botón justo cuando aparece el LED objetivo, un rebote se leería como una pulsación nueva con un tiempo cercano a cero.

La subrutina `wait_release` se ejecuta antes de encender el LED objetivo. Vuelve solo cuando los botones llevan `DEBOUNCE_CYCLES` (20 ms) seguidos sin apretarse: cada lectura con un botón apretado reinicia la cuenta. Así se filtran los rebotes y, además, no se puede dejar un botón apretado antes de que aparezca el objetivo para hacer trampa.

3.4. Medición del tiempo de respuesta

El código 5 muestra el núcleo de la ronda. El cronómetro se inicia con la instrucción siguiente a la que enciende el LED objetivo y se detiene con la instrucción siguiente a la que detecta el botón. Una pulsación con más de un botón a la vez no coincide con el objetivo (que es un solo bit), por lo que cuenta como incorrecta.

```
    sw   a3, LEDS(s0)              # solo el objetivo encendido
    lw   a2, CYCLES(s0)            # en ese mismo instante parte el cronometro
wait_press:
    lw   t0, BUTTONS(s0)
    beqz t0, wait_press            # esperar un boton
    lw   t1, CYCLES(s0)            # fin del cronometro
    bne  t0, a3, wrong             # boton incorrecto (o mas de uno a la vez)
    sub  a0, t1, a2                # ciclos de reaccion (sin signo)
    ...
wrong:
    li   t0, ERROR_CODE
    sw   t0, DISPLAY(s0)           # "EE" y se repite la ronda
    j    round                     # a4 y a5 no cambian
```

Código 5: Encendido del objetivo, medición y botón incorrecto.

El error de la medición se acota con la tabla 1: entre que se escribe el LED y se lee c_0 pasan 2 o 3 ciclos; el bucle `wait_press` da una vuelta cada 6 ciclos (`lw` 3 y `beqz` tomado 3), así que el botón

se detecta a lo más 6 ciclos después de apretarse; y entre la detección y la lectura de c_1 pasan unos 5 ciclos. En total el error es menor que 15 ciclos, es decir, menos de 0,6 μs, más de cien mil veces menor que la resolución de una décima.

3.5. Promedio

El programa acumula en el registro `a4` las décimas de cada ronda correcta. Tras `ROUNDS` respuestas correctas, el promedio se calcula con la misma subrutina de división:

$$\bar{t} = \left\lfloor \frac{\sum_{i=1}^{\text{ROUNDS}} t_i}{\text{ROUNDS}} \right\rfloor \text{ décimas.}$$

Se promedian los tiempos en décimas y no en ciclos porque así el promedio es el de los diez números que el jugador vio en los displays, y porque la suma de diez tiempos en ciclos podría desbordar 32 bits si las rondas fueran lentas. Los displays tienen dos dígitos, así que un tiempo de 10 s o más se muestra como `99`. Esa saturación es solo de la visualización: a la suma entra el valor verdadero. Las rondas con botón incorrecto no modifican ni la suma (`a4`) ni el contador de rondas (`a5`).

4. Generación pseudoaleatoria

El LED objetivo se elige con un LFSR (*linear feedback shift register*) de 32 bits con el polinomio $x^{32} + x^{22} + x^2 + x + 1$, que es de período máximo: recorre los $2^{32} - 1$ estados distintos de cero antes de repetirse [3]. Un paso del LFSR es

$$s \leftarrow \begin{cases} (s \ll 1) \oplus \texttt{0x00400007} & \text{si el bit 31 de } s \text{ es 1,} \\ (s \ll 1) & \text{si no.} \end{cases}$$

La constante `0x00400007` tiene en 1 los bits 22, 2, 1 y 0, que corresponden a los términos del polinomio bajo x^{32}. El código 6 implementa el paso sin desplazamientos: $s \ll 1$ es `add s1, s1, s1`, y "el bit 31 es 1" es lo mismo que "s es negativo" (`bge s1, zero, ...`). Al final, los dos bits bajos del estado eligen el LED: se parte de `1` y se duplica ese número de veces, lo que entrega el LED en formato one-hot (0001, 0010, 0100 o 1000).

```
pt_step:
    add  t0, s1, s1                # t0 = s1 << 1
    bge  s1, zero, pt_no_tap       # bit 31 era 0: sin realimentacion
    xor  t0, t0, t2                # t2 = LFSR_TAPS
pt_no_tap:
    mv   s1, t0
    addi t1, t1, -1
    bnez t1, pt_step               # LFSR_STEPS = 32 veces
```

```
    andi t0, s1, 3                  # indice del LED: 0..3
    li   a0, 1
pt_mask:
    beqz t0, pt_done
    add  a0, a0, a0                 # a0 <<= 1, indice veces
    addi t0, t0, -1
    j    pt_mask
```

Código 6: Paso del LFSR y elección del LED en pick_target.

Semilla. Un LFSR es determinista: con la misma semilla produce siempre la misma secuencia. Si la semilla fuera el contador leído en un punto fijo del programa tras el reset, sería la misma en cada encendido y el juego repetiría la secuencia de LEDs. Por eso el juego espera que el jugador apriete un botón para empezar, y la semilla es el valor del contador de ciclos en ese instante. Un humano no puede repetir un instante con precisión de 40 ns, así que cada partida parte de un estado distinto. Se aplica un `ori` con 1 para que la semilla nunca sea cero, el único estado del que un LFSR no sale.

Por qué 32 pasos por ronda. No todos los bits de la semilla son aleatorios. Los bits altos del contador cambian lento: el bit 30 vale 0 durante los primeros 43 segundos tras el encendido. Los bits bajos tampoco son libres, porque el bucle que lee el botón lo hace cada una cantidad fija de ciclos. Si el LFSR avanzara pocos pasos por ronda, los dos bits que eligen el LED vendrían de los bits poco aleatorios de la semilla. Avanzando 32 pasos, una palabra completa, cada bit del estado pasa a depender de los bits medios de la semilla, que sí dependen de cuándo apretó el jugador.

Esto se comprobó con un modelo del LFSR en Python (`sim/lfsr_modelo.py`), que simula 20.000 partidas con semillas como las que produce el programa: apretones entre 0,5 y 5 segundos tras el encendido, leídos cada P ciclos (tabla 4). Con 2 pasos por ronda, el primer LED es *siempre* el mismo. Con 32 pasos, cada LED sale cerca del 25 % de las veces y aparecen los 16 pares posibles de LEDs consecutivos.

Cuadro 4: Modelo de 20.000 partidas. P: cada cuántos ciclos lee el programa los botones.

Pasos por ronda	P	Primer LED: 0 / 1 / 2 / 3 (%)	Pares distintos	Secuencias distintas
2	1	100 / 0 / 0 / 0	1 de 16	14 133
2	16	100 / 0 / 0 / 0	1 de 16	14 218
32	1	25,5 / 24,9 / 25,0 / 24,6	16 de 16	19 809
32	16	24,7 / 25,2 / 25,1 / 25,0	16 de 16	19 796

Fuente: Elaboración propia.

Cada paso toma 12 o 13 ciclos según si hay realimentación, así que los 32 pasos cuestan unos 400 ciclos (16 μs). Ocurren antes de encender el objetivo y por lo tanto no afectan la medición.

5. El programa en assembly

El juego completo está en `sw/game.s`. Ocupa 114 palabras de máquina (456 de los 2048 bytes de la RAM) y no necesita stack.

5.1. Estructura

El programa tiene un cuerpo principal con dos niveles, partida y ronda, y seis subrutinas (tabla 5).

1. `_start`: carga la base de periféricos en `s0` y pone los displays en `00`.
2. `game`: espera que se suelten los botones, parpadea hasta que se apriete uno (`wait_start`), toma ese instante como semilla del LFSR y reinicia el contador de rondas (`a5`) y la suma de tiempos (`a4`).
3. `round`: enciende los cuatro LEDs, espera `WAIT_CYCLES`, espera botones sueltos, elige el objetivo (`pick_target`), lo enciende y lee el contador.
4. `wait_press`: espera un botón y lee el contador otra vez. Si el botón es incorrecto salta a `wrong`. Si es correcto, convierte ciclos a décimas, suma, muestra el tiempo e incrementa `a5`.
5. Si `a5` $<$ `ROUNDS`, vuelve a `round`. Si no, apaga los LEDs, deja ver el último tiempo durante `WAIT_CYCLES`, muestra el promedio y vuelve a `game`, donde `wait_start` lo mantiene en pantalla mientras los LEDs parpadean.

Cuadro 5: Subrutinas de `game.s`. Todas son hoja: no llaman a otras.

Subrutina	Entrada / salida	Qué hace
`wait_cycles`	`a0` = ciclos	espera `a0` ciclos leyendo el contador
`wait_release`	–	vuelve cuando los botones llevan `DEBOUNCE_CYCLES` sin apretarse
`wait_start`	sale `a0` = contador	parpadea 0101/1010 hasta que se apriete un botón
`pick_target`	sale `a0` = LED one-hot	avanza el LFSR (`s1`) 32 pasos y elige un LED
`divu`	`a0`, `a1` $\rightarrow$ `a0`	división entera sin signo por restas sucesivas
`show_number`	`a0` = número	muestra `a0` en decimal en los displays, máximo 99

Fuente: Elaboración propia.

5.2. Uso de registros

RV32E tiene solo 16 registros, así que se asignaron de forma fija (tabla 6). Como ninguna subrutina llama a otra, el registro de retorno `ra` nunca se sobrescribe antes de un `ret`, y el programa no necesita stack.

Cuadro 6: Asignación de registros.

Registro	Contenido	Vida
`s0`	base de periféricos `0x80000000`	todo el programa
`s1`	estado del LFSR	una partida
`a2`	contador al encender el objetivo (c_0)	una ronda
`a3`	LED objetivo en one-hot	una ronda
`a4`	suma de décimas de las rondas correctas	una partida
`a5`	rondas correctas	una partida
`a0`, `a1`	argumentos y resultado de subrutinas	una llamada
`t0`–`t2`	temporales	una llamada

Fuente: Elaboración propia.

5.3. Aritmética sin multiplicación, división ni desplazamientos

- **Multiplicar por 2** es sumar un número consigo mismo: `add x, x, x`. Para escribir las decenas en los 4 bits altos del display se multiplican por 16 con cuatro sumas seguidas.
- **Desplazar un bit a la izquierda** es multiplicar por 2. Así avanza el LFSR y así se construye la máscara del LED objetivo.
- **Dividir** es contar cuántas veces cabe el divisor restándolo (`divu`). Es lento para cocientes grandes, pero aquí son pequeños: 99 como máximo.
- **Separar decenas y unidades** para los displays es otra división por restas: se resta 10 mientras se pueda, y lo que queda son las unidades.
- **Leer el bit 31** no requiere desplazar: es el bit de signo, que vale 1 exactamente cuando el registro es negativo.

```
divu:                               # a0 = a0 / a1 (sin signo, trunca)
    li   t0, 0                      # t0 = cociente
du_loop:
    bltu a0, a1, du_done
    sub  a0, a0, a1
    addi t0, t0, 1
    j    du_loop
du_done:
    mv   a0, t0
    ret

show_number:                        # muestra a0 en decimal (maximo 99)
    li   t0, 99
    bleu a0, t0, sn_fits
    mv   a0, t0
sn_fits:
    li   t1, 0                      # t1 = decenas
    li   t2, 10
sn_tens:
    bltu a0, t2, sn_write           # lo que queda son las unidades
    sub  a0, a0, t2
    addi t1, t1, 1
    j    sn_tens
sn_write:
    add  t1, t1, t1                 # decenas * 2
    add  t1, t1, t1                 #         * 4
    add  t1, t1, t1                 #         * 8
    add  t1, t1, t1                 #         * 16
    add  t1, t1, a0                 # + unidades
    sw   t1, DISPLAY(s0)
    ret
```

Código 7: Subrutinas divu y show_number.

5.4. Instrucciones utilizadas

La tabla 7 lista las instrucciones de máquina que genera el assembler para `game.s`, después de expandir las pseudo-instrucciones. El programa usa 17 instrucciones distintas de RV32I y ninguna fuera de RV32E. No usa desplazamientos.

Cuadro 7: Instrucciones de máquina de `game.hex` (114 en total).

Instrucción	Veces	Uso principal
`addi`	32	constantes (`li`), copias (`mv`), contadores
`jal`	18	llamadas (`call`) y saltos (`j`)
`lw`	12	leer el contador de ciclos y los botones
`add`	9	multiplicar por 2 (máscara, LFSR, decenas) y acumular
`lui`	7	parte alta de constantes grandes (`li`)
`sub`	6	intervalos de tiempo y divisiones por restas
`bltu`	6	comparaciones sin signo de tiempos y cocientes
`jalr`	6	retorno de subrutina (`ret`)
`sw`	6	escribir LEDs y displays
`beq`, `bne`	3, 3	`beqz`/`bnez` y comparación del botón con el objetivo
`bge`, `bgeu`	1, 1	bit 31 del LFSR; saturación a 99 (`bleu`)
`andi`, `ori`	1, 1	índice del LED; semilla distinta de cero
`xor`, `xori`	1, 1	realimentación del LFSR; alternar el parpadeo

Fuente: Elaboración propia.

6. El assembler

El assembler es el programa `asm/pochoasm.py`, escrito en Python sin bibliotecas externas. Recibe un archivo de assembly y escribe un archivo `.hex` con una palabra de 32 bits en hexadecimal por línea. La palabra de la línea k queda en la dirección $4k$ de la RAM, que es lo que espera `$readmemh` en `pochoco_ram.v`.

```
python3 asm/pochoasm.py sw/game.s -o sw/game.hex [-l sw/game.lst] [-D NOMBRE=VALOR]
```

Código 8: Uso del assembler.

La opción `-l` escribe además un listado con la dirección, la palabra y la línea de origen de cada instrucción. La opción `-D` reemplaza el valor de una constante `.equ` sin editar el archivo; se usa para simular el juego con tiempos cortos (sección 7).

6.1. Sintaxis aceptada

Cada línea tiene la forma

$$[\,\textit{etiqueta}\texttt{:}\,]\ \ [\,\textit{instrucción}\ \ \textit{operando}\texttt{,}\ \textit{operando}\ldots\,]\ \ [\,\texttt{\#}\ \textit{comentario}\,]$$

y cualquiera de las tres partes puede faltar. Una línea puede tener varias etiquetas.

- **Registros**: `x0` a `x15` o sus nombres ABI (`zero`, `ra`, `sp`, `gp`, `tp`, `t0`–`t2`, `s0`/`fp`, `s1`, `a0`–`a5`). Los registros que existen en RV32I pero no en RV32E (`x16`–`x31`, `a6`, `s2`, `t3`, ...) producen un error explicativo.
- **Números**: decimales, hexadecimales (`0x`), binarios (`0b`), con signo opcional. También se acepta el nombre de una constante `.equ`, con signo menos opcional.
- **Acceso a memoria**: `desplazamiento(registro)`, por ejemplo `lw t0, CYCLES(s0)`. Si el desplazamiento se omite vale 0.
- **Destinos de salto**: siempre una etiqueta.
- **Instrucciones**: las 40 de RV32I (aritmético-lógicas con registros y con inmediato, desplazamientos, cargas, almacenamientos, branches, `lui`, `auipc`, `jal`, `jalr`, `ecall`, `ebreak` y `fence`).
- **Directivas**: `.equ NOMBRE, valor` define una constante. `.text`, `.section`, `.global` y `.globl` se aceptan y se ignoran: hay un solo programa en una sola memoria desde la dirección 0.

Las pseudo-instrucciones son abreviaturas estándar de RISC-V [1] que no existen en el hardware. El assembler las reescribe como instrucciones reales (tabla 8).

Cuadro 8: Pseudo-instrucciones aceptadas y su traducción.

Pseudo-instrucción	Instrucción real
`nop`	`addi zero, zero, 0`
`mv rd, rs`	`addi rd, rs, 0`
`not rd, rs`	`xori rd, rs, -1`
`neg rd, rs`	`sub rd, zero, rs`
`li rd, valor`	`addi`, `lui` o `lui` + `addi`
`j etiq`	`jal zero, etiq`
`jal etiq`, `call etiq`	`jal ra, etiq`
`jr rs`	`jalr zero, 0(rs)`
`ret`	`jalr zero, 0(ra)`
`beqz rs, etiq` / `bnez rs, etiq`	`beq` / `bne rs, zero, etiq`
`bgt a, b, etiq` / `ble a, b, etiq`	`blt` / `bge b, a, etiq`
`bgtu a, b, etiq` / `bleu a, b, etiq`	`bltu` / `bgeu b, a, etiq`

Fuente: Elaboración propia.

En el assembler de GNU, `call` se traduce como `auipc` + `jalr` para alcanzar cualquier dirección de 32 bits. Aquí se traduce como un solo `jal`, cuyo alcance de ±1 MB sobra para una RAM de 2 KB.

6.2. Tratamiento de etiquetas: tres pasadas

El problema central de un assembler son las etiquetas usadas antes de definirse: en `call show_number`, la subrutina está al final del archivo y todavía no se sabe en qué dirección quedará. Por eso el texto se recorre tres veces:

0. **Constantes.** Se separa cada línea en etiquetas, mnemónico y operandos, y se evalúan las `.equ` en orden.

1. **Direcciones.** Se recorre el programa con un contador de posición que parte en 0. Cada etiqueta recibe el valor actual del contador y cada instrucción lo avanza en 4 bytes. Las pseudo-instrucciones se expanden aquí porque determinan cuántas palabras ocupa cada línea: `li` puede ocupar una o dos.

2. **Codificación.** Con todas las direcciones conocidas, cada instrucción real se traduce a sus 32 bits.

Los branches y `jal` no guardan la dirección de destino sino la distancia desde la instrucción que salta: desplazamiento = dirección de la etiqueta $-$ *pc*. Un desplazamiento negativo es un salto hacia atrás, como el de un bucle. Como el tamaño de `li` se decide en la primera pasada, `li` solo acepta números y constantes `.equ`, no etiquetas.

6.3. Codificación e inmediatos

RISC-V tiene seis formatos de instrucción [1, 4] (tabla 9). Todos dejan el opcode en los bits 6:0 y los registros siempre en las mismas posiciones: `rd` en 11:7, `rs1` en 19:15 y `rs2` en 24:20. Lo que cambia es dónde se reparte el inmediato. Los formatos B y J no guardan el bit 0 del desplazamiento, porque las instrucciones están alineadas y siempre es 0.

Cuadro 9: Formatos de instrucción de RV32I, del bit 31 al 0.

Formato	Distribución de los bits
R	funct7 | rs2 | rs1 | funct3 | rd | opcode
I	imm[11:0] | rs1 | funct3 | rd | opcode
S	imm[11:5] | rs2 | rs1 | funct3 | imm[4:0] | opcode
B	imm[12] imm[10:5] | rs2 | rs1 | funct3 | imm[4:1] imm[11] | opcode
U	imm[31:12] | rd | opcode
J	imm[20] imm[10:1] imm[11] imm[19:12] | rd | opcode

Fuente: Elaboración propia.

Cada formato es una función de una línea que coloca los campos con desplazamientos y máscaras (código 9). La información de cada instrucción está en una tabla de datos: el mnemónico apunta a su formato, opcode, funct3 y funct7. Por ejemplo, `'bne': ('B', 0x63, 1, 0)`. Agregar una instrucción es agregar una fila.

```
def enc_i(imm, rs1, f3, rd, op):
    return (imm & 0xFFF) << 20 | rs1 << 15 | f3 << 12 | rd << 7 | op

def enc_b(off, rs2, rs1, f3, op):
    # imm[12|10:5] en 31:25, imm[4:1|11] en 11:7 (el bit 0 no se guarda)
    return (((off >> 12) & 1) << 31 | ((off >> 5) & 0x3F) << 25 | rs2 << 20 | rs1 << 15
            | f3 << 12 | ((off >> 1) & 0xF) << 8 | ((off >> 11) & 1) << 7 | op)
```

Código 9: Codificación de los formatos I y B en pochoasm.py.

Cada inmediato se verifica contra lo que cabe en su campo, con signo: $[-2048, 2047]$ para los formatos I y S, $[0, 31]$ para la cantidad de desplazamiento, $[0, 2^{20}-1]$ para `lui` y `auipc`, $[-4096, 4094]$ para los branches y $[-2^{20}, 2^{20}-2]$ para `jal`.

La pseudo-instrucción `li`. Carga un valor de 32 bits en un registro. Si cabe en 12 bits con signo, basta un `addi` desde `zero`. Si no, se parte en una parte alta de 20 bits (`lui`) y una baja de 12 bits (`addi`). La dificultad es que `addi` interpreta sus 12 bits *con signo*: si el bit 11 de la parte baja es 1, `addi` resta en vez de sumar, y la parte alta debe compensarlo sumando 1. El assembler calcula

$$\text{lo} = \text{extensión de signo de } (v \,\&\, \texttt{0xFFF}), \qquad \text{hi} = \frac{v - \text{lo}}{2^{12}} \bmod 2^{20}.$$

Si $\text{lo} = 0$ basta el `lui`.

6.4. Validación

Cada error se reporta con el archivo, la línea y el texto de la línea, y el assembler sigue revisando el resto del archivo para mostrar todos los errores de una vez. Si hay algún error no escribe el `.hex`, para que nunca quede en disco un archivo a medio generar (código 10).

```
malo.s:3: error: 'x16' no existe en RV32E: solo hay 16 registros (x0-x15)
    add  x16, a0, a1
malo.s:7: error: inmediato de 12 bits fuera de rango: 3000 (permitido -2048 a 2047)
    addi t0, t0, 3000
malo.s:9: error: etiqueta no definida: 'fin'
    j    fin
3 error(es); no se escribio ningun archivo.
```

Código 10: Salida del assembler ante un programa con errores.

Los errores detectados son: registro inexistente o fuera de RV32E, instrucción desconocida, cantidad incorrecta de operandos, operando vacío, inmediato o salto fuera de rango, acceso a memoria mal escrito, constante no definida, etiqueta no definida o repetida, constante repetida, nombre de constante reservado, directiva no soportada y programa más grande que la RAM de 512 palabras. Además, el assembler *avisa* (sin fallar) si el programa usa un desplazamiento, porque el Espino Core lo ejecutaría como una suma.

6.5. Ejemplos de traducción

Un branch. La instrucción `bne t0, a3, wrong` de `game.s` está en la dirección `0x5C`, y la etiqueta `wrong` quedó en `0xA8`.

1. Desplazamiento: `0xA8` − `0x5C` = `0x4C` = 76. En 13 bits: `0 0000 0100 1100`.
2. Campos: `bne` es formato B con opcode `1100011` y funct3 `001`; `t0` es `x5` (`00101`) y `a3` es `x13` (`01101`).
3. Se reparte el desplazamiento: imm[12] = 0, imm[10:5] = `000010`, imm[4:1] = `0110`, imm[11] = 0.

imm[12]	imm[10:5]	rs2	rs1	funct3	imm[4:1]	imm[11]	opcode
0	000010	01101	00101	001	0110	0	1100011

Agrupando de a cuatro bits, `0000 0100 1101 0010 1001 0110 0110 0011` = `0x04D29663`, que es la línea 24 de `game.hex`.

Una constante grande. `li a0, WAIT_CYCLES`, con WAIT_CYCLES = 75 000 000 = `0x047868C0`:

1. Los 12 bits bajos son `0x8C0`. Su bit 11 es 1, así que con signo valen `0x8C0` − 4096 = −1856.
2. La parte alta se redondea hacia arriba para compensar: (75 000 000 + 1856)/4096 = 18 311 = `0x4787`.
3. `lui a0, 0x4787` carga `0x04787000` y `addi a0, a0, -1856` deja 75 000 000.

Codificadas: `lui` (formato U, opcode `0110111`, rd = `01010`) da `0x04787537` y `addi` (formato I, inmediato `0x8C0`, rs1 = rd = `01010`, opcode `0010011`) da `0x8C050513`. Son las líneas 13 y 14 de `game.hex`.

7. Verificación

La verificación tiene cuatro niveles en simulación, de lo más aislado a lo más completo, y termina en la placa.

7.1. Pruebas del assembler

El archivo `asm/test_pochoasm.py` contiene 20 pruebas con `unittest` (tabla 10), que se ejecutan con `make test`. Las dos primeras filas son las más importantes, porque comparan contra algo que no escribió el grupo.

Cuadro 10: Pruebas del assembler.

Grupo	Pruebas	Qué se comprueba
Referencia GNU	1	los 3 ejemplos del repositorio original, ensamblados por GNU, palabra por palabra
Ida y vuelta	5	todos los inmediatos de I, S, B y J codificados y decodificados con las fórmulas de `espino_decoder.v`; saltos con etiquetas hacia adelante y hacia atrás
Palabras conocidas	9	`nop`, `ret`, `ecall` y otras palabras de la especificación; todas las pseudo-instrucciones; `li` con miles de valores incluidos los bordes; `.equ`; `-D`; aviso de desplazamientos
Validación	4	programas erróneos, cada uno con su mensaje y su línea; reporte de todos los errores juntos
El juego	1	`game.s` se ensambla sin avisos y cabe en la RAM

Fuente: Elaboración propia.

7.2. Prueba de instrucciones sobre el procesador

El programa `sim/isa_test.s` ejercita las instrucciones de RV32I en 22 pruebas. Cada prueba compara su resultado con el esperado y, si falla, escribe su número en los displays. Se ensambla con el assembler del grupo y se ejecuta sobre el RTL completo del SoC en Icarus Verilog (`sim/isa_tb.v`). Así se prueba el assembler contra el hardware real: si una instrucción se codificara mal, el procesador ejecutaría otra cosa. El resultado esperado es `AA` en los displays y los LEDs en 1111.

7.3. Simulación del juego

El testbench `sim/game_tb.v` juega dos partidas completas contra el SoC corriendo `game.s` y comprueba cada regla del enunciado mirando solo los pines de la placa: los cuatro LEDs y los catorce

segmentos de los displays, que decodifica de vuelta a dígitos como los leería una persona. Aprieta y suelta los botones con rebotes (tres pulsos cortos antes de quedar firmes).

Simular 3 segundos reales son 75 millones de ciclos, demasiado para un simulador. Por eso el testbench usa el mismo `game.s` ensamblado con constantes más cortas mediante `-D`: `WAIT_CYCLES` = 20.000, `TENTH_CYCLES` = 1.000, `DEBOUNCE_CYCLES` = 200 y `BLINK_CYCLES` = 2.000. El programa es idéntico; solo cambian esos cuatro números.

La primera partida usa diez tiempos de reacción elegidos para ejercitar los casos borde: 3, 7, 12, 5, 120, 1, 15, 4, 8 y 6 décimas. Los displays deben mostrar `03 07 12 05 99 01 15 04 08 06`: un dígito, dos dígitos y una saturación a 99, con la suma completa (181) entrando al promedio $\lfloor 181/10 \rfloor = 18$. En la ronda 3 se aprieta primero un botón incorrecto (deben aparecer `EE` y la repetición de la ronda) y en la ronda 7 se mantiene un botón apretado durante toda la fase de los cuatro LEDs (no debe ser un error, y el objetivo debe esperar a que se suelte). La tabla 11 resume las revisiones.

Cuadro 11: Revisiones del testbench del juego (`make sim`).

Regla comprobada
Al encender: displays en `00` y LEDs alternando 0101 y 1010
Los cuatro LEDs duran al menos `WAIT_CYCLES` en cada ronda, y después queda un solo LED
El tiempo mostrado es el esperado para cada reacción (incluida la saturación a 99)
Botón incorrecto: `EE` y la ronda se repite; no cuenta como ronda ni suma al promedio
Botón sostenido durante los cuatro LEDs: no es error y el objetivo espera al menos `DEBOUNCE_CYCLES` sin botones
Tras la décima ronda el último tiempo sigue visible `WAIT_CYCLES`; luego se muestra el promedio con LEDs parpadeando
Una partida nueva empieza en `00`
Dos partidas empezadas en instantes distintos producen secuencias de LEDs distintas

Fuente: Elaboración propia.

7.4. Simulación post-síntesis

Las simulaciones anteriores corren sobre el RTL, la descripción en Verilog. Lo que se graba en la FPGA es el circuito que produce Yosys, hecho de tablas de búsqueda, flip-flops y bloques de RAM de la iCE40, con el programa ya cargado en esos bloques. `make sim-post` sintetiza el diseño con la versión corta del juego, exporta el circuito resultante como Verilog de celdas y le aplica el mismo testbench. Tarda unos 7 minutos.

7.5. Prueba en la placa: el reset de encendido

Con las simulaciones aprobadas, el juego se programó en la Go Board y no funcionó: los LEDs no se encendían nunca. Como la simulación no reproducía la falla, se diagnosticó directamente en la placa, grabando programas de prueba y observando los LEDs. Los programas cortos (7 a 18 palabras, e incluso uno de 128 palabras cuyo relleno nunca se ejecutaba) funcionaban; `game.s` (114 palabras) fallaba, incluso con marcas en los LEDs y con la RAM en otra configuración. Se descartaron, en orden:

- **Temporización.** nextpnr e `icetime` dan más de 42 MHz, y el bitstream de un programa que funciona y el de uno que falla resultaron idénticos salvo el contenido inicial de la RAM.
- **Traducción del bitstream.** Se reconstruyó en Verilog el bitstream que falla con `icebox_vlog` y se simuló: el procesador enciende la primera marca en el ciclo 22.
- **Modo de los bloques de RAM.** Con 256 palabras (otro modo de bloque) el juego siguió fallando.
- **Dependencias entre instrucciones.** Un `sw` que usa el registro escrito por la instrucción anterior funcionó en la placa.

La explicación que calza con todos los casos es el tiempo que tarda la RAM en estar disponible tras la configuración. El SoC original suelta el reset a los 15 ciclos (0,6 μs). Si en ese momento los bloques de RAM todavía no entregan su contenido, el procesador recibe ceros, que decodifica como instrucciones sin efecto, y cuando la RAM empieza a responder está en la mitad del programa con todos los registros en cero; en particular `s0` no contiene la dirección de los periféricos. Los programas cortos sobreviven porque el contador de programa da la vuelta al llegar a los 2 KB de la RAM y el programa vuelve a empezar desde la dirección 0, esta vez con la RAM disponible.

La corrección fue mantener al procesador en reset durante 4.095 ciclos (código 3). Con ese cambio, y sin ningún otro, el juego funcionó en la placa. Ninguna simulación podía detectar el problema, porque los modelos de simulación de los bloques de RAM entregan su contenido desde el primer ciclo.

8. Compilación, programación y cambio de constantes

Todas las herramientas vienen en OSS CAD Suite [6]: Yosys, nextpnr, IceStorm, Icarus Verilog y Python 3. Desde su terminal, en la carpeta del proyecto, se usan los comandos de la tabla 12.

Cuadro 12: Comandos del Makefile.

Comando	Qué hace
`make prog`	ensambla `game.s`, sintetiza, hace place and route, genera `game_top.bin` y programa la Go Board
`make`	lo mismo, sin programar
`make hex`	solo `sw/game.s` → `sw/game.hex`
`make test`	pruebas del assembler
`make sim`	prueba de instrucciones y simulación del juego
`make sim-post`	el mismo juego sobre el circuito sintetizado

Fuente: Elaboración propia.

El Makefile conoce las dependencias: `game_top.bin` depende de `game.hex`, que depende de `game.s` y del assembler. Tras editar `game.s` basta `make prog`. Para regenerar `game.hex` sin make:

```
python3 asm/pochoasm.py sw/game.s -o sw/game.hex
```

Cambiar una constante. Todas las constantes están con nombre al comienzo de `sw/game.s` (tabla 3). Por ejemplo, para jugar cinco rondas en vez de diez se cambia `.equ ROUNDS, 10` por `.equ ROUNDS, 5` y se ejecuta `make prog`. El promedio se sigue calculando bien porque divide por `ROUNDS` y no por 10. Del mismo modo, `.equ WAIT_CYCLES, 125000000` deja la fase de los cuatro LEDs en 5 segundos. El `.hex` cambia porque la RAM se inicializa al sintetizar, así que un cambio en el programa exige sintetizar el diseño completo.

9. Conclusiones

Se implementó el juego de reflejos completo sobre el Pochoco SoC: espera de tres segundos, elección pseudoaleatoria del LED objetivo, medición del tiempo de respuesta en décimas de segundo, manejo de botones incorrectos y promedio de diez rondas. El diseño sintetiza para la iCE40 HX1K con margen de frecuencia (42,8 MHz frente a 25 MHz) y el juego funciona en la Go Board.

El único cambio de hardware necesario para el juego fue un contador de ciclos de 32 bits. Con él, toda medición de tiempo se reduce a una resta sin signo que además tolera que el contador dé la vuelta. La alternativa, bucles de espera calibrados, habría acoplado la duración de cada fase al número exacto de instrucciones del programa.

Las restricciones del procesador obligaron a expresar operaciones elementales con las instrucciones disponibles: multiplicar por dos es sumar un número consigo mismo, leer el bit 31 es preguntar si el número es negativo y dividir es restar repetidamente. El modelo del generador pseudoaleatorio mostró que el número de pasos entre elecciones importa: con pocos pasos el primer LED de cada partida era siempre el mismo, aunque la semilla viniera del contador.

La prueba en la placa mostró el límite de la simulación: el reset de encendido original soltaba al procesador antes de que la RAM estuviera disponible, algo que los modelos de simulación no representan. La causa se aisló grabando programas de prueba cada vez más cercanos al juego y descartando una hipótesis por vez, y la corrección fue una línea de Verilog. En el assembler, el problema de fondo son las referencias hacia adelante, que se resolvieron con pasadas separadas para las direcciones y para la codificación, y la decisión que más confianza dio fue probarlo contra referencias externas: el assembler de GNU, las fórmulas del decodificador del procesador y el procesador mismo en simulación.

Referencias

[1] A. Waterman, K. Asanović et al., *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, RISC-V International, 2024. https://docs.riscv.org/reference/isa/

[2] N. Villegas, *Pochoco SoC & Espino Core*, repositorio GitHub, Universidad de los Andes, 2026. https://github.com/nic0villegasc/pochoco_soc

[3] P. Alfke, *Efficient Shift Registers, LFSR Counters, and Long Pseudo-Random Sequence Generators*, Xilinx, Application Note XAPP 052, 1996.

[4] D. A. Patterson y J. L. Hennessy, *Computer Organization and Design RISC-V Edition*, 2.ª ed., Morgan Kaufmann, 2020.

[5] Nandland, *The Go Board: FPGA Development Board*. https://nandland.com/the-go-board/

[6] YosysHQ, *OSS CAD Suite*. https://github.com/YosysHQ/oss-cad-suite-build

Declaración de Uso de Herramientas de Inteligencia Artificial

1. Identificación de la actividad

- **Curso:** Arquitectura de Computadores
- **Código del curso:** 433
- **Nombre de la actividad:** Proyecto Nº2: Juego en Assembly sobre Pochoco SoC
- **Fecha de entrega:** Martes 13 de octubre de 2026
- **Nombre(s) de(los) estudiante(s):** Josefa Alcaíno Lira, Felipe Izcúe Ferrer, Martín Norton Haigis

2. Declaración sobre uso de IA

☐ No se utilizaron herramientas de inteligencia artificial en ninguna etapa del desarrollo de esta actividad.
☒ Sí se utilizaron herramientas de inteligencia artificial, según se detalla en la sección 3 de este documento.

3. Descripción de uso de IA

Herramienta empleada	Claude (Anthropic): Sonnet 5.5 (Medium), Opus 5 (Ultracode)
Fechas de uso	13 de octubre de 2026
Estudiantes	Josefa Alcaíno Lira, Felipe Izcúe Ferrer y Martín Norton Haigis
Propósito del uso	Creación de códigos en lenguaje Verilog, Assembly (RV32E) y Python. Explicación de códigos y funcionamiento de la placa en general para apoyo en redacción del informe y del repositorio. Apoyo en corrección de formato además de ortografía y puntuación tanto en el repositorio como en el informe. Apoyo con la escritura del informe en LaTeX a partir del repositorio del proyecto, y revisión del código del repositorio para describirlo.
Descripción de integración	Se utlizó Claude para la creación del proyecto, por la parte de la configuración del juego, tanto como apoyo fundamental para la escritura, desarrollo y corrección del informe como del repositorio de GitHub.

4. Declaración de responsabilidad académica

Los firmantes de este documento declaramos que:

- Hemos revisado críticamente todo el contenido entregado.
- Comprendemos plenamente el trabajo presentado y somos capaces de explicarlo y defenderlo académicamente.
- Asumimos responsabilidad por la calidad, veracidad y originalidad del contenido entregado, independientemente del uso o no uso de herramientas de inteligencia artificial.
- La información declarada en este documento es veraz.







Josefa Alcaíno Lira	Felipe Izcúe Ferrer	Martín Norton Haigis